

目录

第 1 节	实验背景与数据集	1
1	实验背景	1
2	数据集	1
3	数据预处理	1
1	字符索引转换	1
2	序列分割	1
3	词汇表构建	1
第 2 节	实验目的	1
第 3 节	实验方法	2
1	网络结构设计	2
2	网络结构示意图	2
3	损失函数	2
4	优化方法	2
1	优化器选择	2
2	学习率调度	3
3	梯度裁剪	3
5	生成策略	3
6	超参数配置	3
第 4 节	实验结果	3
1	训练过程可视化	3
2	训练过程分析	5
3	生成结果示例	5
第 5 节	实验分析与总结	5
1	模型性能分析	5
2	参数影响分析	5
3	损失函数与评价指标	5
1	交叉熵损失	5
2	困惑度 (Perplexity)	6
4	优化策略效果	6
5	实验总结	6
第 6 节	附录	6
1	核心代码模块	6
2	模型定义代码	7
3	TensorBoard 日志记录	7

第 1 节 实验背景与数据集

1 实验背景

本实验构建了一个基于 RNN 的诗歌生成模型。通过训练循环神经网络学习诗歌文本中的字符序列规律，使模型能够根据给定的起始字符自动生成符合古诗风格的文本。

2 数据集

本实验使用古诗数据集：

- 数据集格式：纯文本文件（poetry.txt）
- 数据内容：中文古诗文本
- 序列长度：20 个字符
- 最大词汇量：10000 个字符

3 数据预处理

1 字符索引转换

构建字符到整数的映射表（word_to_int）和整数到字符的映射表（int_to_word），将文本转换为数值序列。

2 序列分割

将长文本分割为固定长度（n_step=20）的序列，每个输入序列的标签为后移一位的字符序列（最后一个位置的标签为第一个字符）。

3 词汇表构建

统计字符频率，保留频率最高的 max_vocab 个字符，其余字符映射为统一标记。

第 2 节 实验目的

- 理解循环神经网络（RNN）处理序列数据的原理
- 掌握字符级语言模型（Character-Level Language Model）的构建方法
- 学会使用 PyTorch 实现 RNN 并进行诗歌生成
- 掌握 TensorBoard 可视化训练过程的方法
- 理解困惑度（Perplexity）作为语言模型评价指标的含义

第 3 节 实验方法

1 网络结构设计

本实验构建了一个基于 RNN 的字符级语言模型。

- **嵌入层**：将字符索引映射为 `embed_dim` 维向量
- **Dropout 层**：防止过拟合，dropout 率 0.5
- **RNN 层**：2 层 RNN，隐藏层大小 256
- **线性投影层**：将隐藏状态映射到词汇表大小的输出

2 网络结构示意图

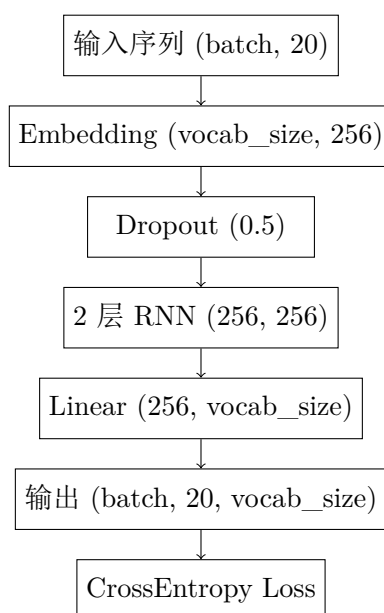


图 3.1 RNN 诗歌生成模型结构

3 损失函数

交叉熵损失：

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij}) \quad (3.1)$$

其中 N 为样本数， C 为类别数（词汇表大小）。

4 优化方法

1 优化器选择

- 默认使用 Adam 优化器

- 学习率： 1×10^{-4}
- 权重衰减（L2 正则化）： 1×10^{-5}

2 学习率调度

可采用 StepLR（每 5 个 epoch 衰减为原来的 0.5）、CosineAnnealingLR 或 ReduceLROnPlateau 策略。

3 梯度裁剪

为防止 RNN 梯度爆炸问题，使用梯度裁剪：

$$\|\nabla W\| \leq 5 \quad (3.2)$$

5 生成策略

预测阶段采用 Top-N 随机采样策略：从前 N 个概率最高的字符中按概率随机选择一个作为一个字符，避免模型生成重复内容。

6 超参数配置

参数	取值
embed_dim	256
hidden_size	256
num_layers	2
n_step（序列长度）	20
max_vocab	10000
batch_size	128
learning_rate	1×10^{-4}
dropout	0.5
grad_clip	5
top_n	5
epochs	20

表 3.1 超参数设置

第 4 节 实验结果

1 训练过程可视化

训练过程中的各项指标变化曲线如图所示。

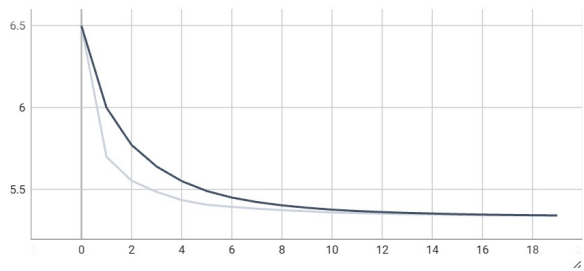


图 4.1 Epoch 平均 Loss 曲线

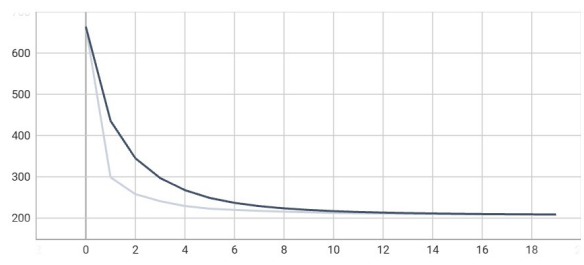


图 4.2 Perplexity（困惑度）曲线

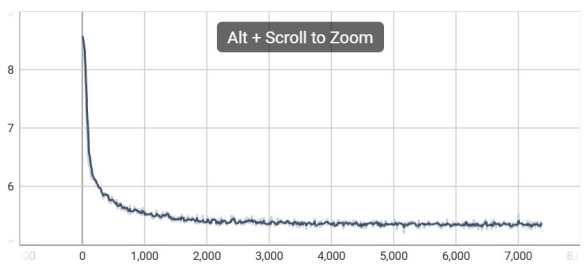


图 4.3 Batch Loss 曲线

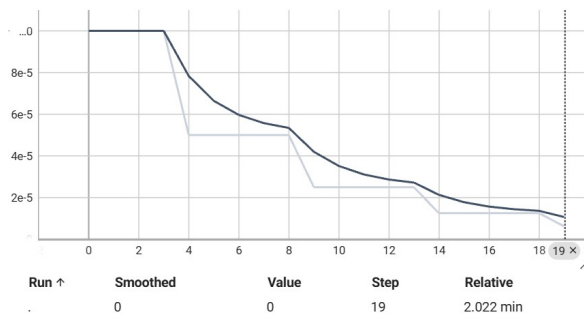


图 4.4 Learning Rate 曲线

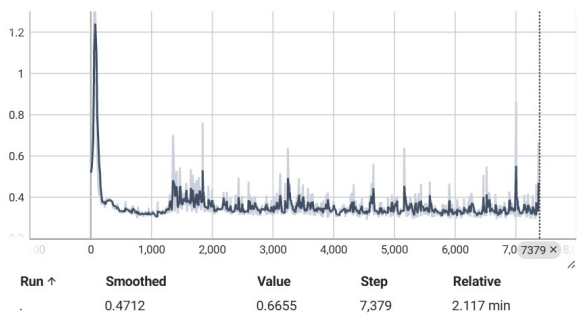


图 4.5 Gradient 范数变化曲线

2 训练过程分析

- **Loss 曲线:** 训练过程中 Loss 逐渐下降并趋于稳定，表明模型在学习诗歌文本的字符规律
- **Batch Loss:** 每个 batch 的损失存在波动，但整体呈下降趋势，说明每个批次训练都在进行有效学习
- **Perplexity:** 困惑度随训练逐渐降低，说明模型对下一个字符的预测越来越准确。Perplexity 越低，模型生成的文本越流畅
- **Learning Rate:** 学习率按调度策略逐步衰减，后期较小的学习率有助于模型稳定收敛
- **Gradient 范数:** 梯度范数在合理范围内波动，说明梯度裁剪有效防止了梯度爆炸问题

3 生成结果示例

模型训练完成后，以自定义字开头生成诗歌示例（在 TensorBoard Text 标签页中可查看各 epoch 生成质量的变化）。

第 5 节 实验分析与总结

1 模型性能分析

- RNN 能够有效学习字符序列的规律，生成的诗歌文本具有一定的古诗风格
- 两层 RNN 结构提供了足够的表达能力，同时不会导致过度复杂
- Embedding 层将离散字符映射为连续向量，有助于捕捉字符间的语义关系

2 参数影响分析

参数	影响
embed_dim	影响字符表示能力，过大易过拟合，过小表达能力不足
hidden_size	影响 RNN 记忆能力，越大记忆越长但计算量增加
num_layers	层数增加表达能力增强，但可能导致梯度消失
n_step	序列长度影响模型学习的上下文范围
dropout	防止过拟合，值过大可能导致欠拟合
top_n	影响生成文本的多样性与质量之间的平衡
learning_rate	过大导致训练不稳定，过小收敛缓慢

表 5.1 参数影响分析

3 损失函数与评价指标

1 交叉熵损失

用于衡量模型预测的字符概率分布与真实标签之间的差异，是字符级语言模型的标准损失函数。

2 困惑度 (Perplexity)

$$\text{Perplexity} = e^{\text{Loss}} \quad (5.1)$$

困惑度可以理解为：模型在预测下一个字符时，平均需要在多少个候选字符中进行选择。困惑度越低，模型越“自信”，生成质量越高。

4 优化策略效果

- **梯度裁剪**：有效防止 RNN 训练中的梯度爆炸问题，保持训练稳定
- **Dropout**：减少过拟合，提高模型泛化能力
- **学习率调度**：逐步降低学习率有助于模型在后期精细调整参数
- **权重衰减**：L2 正则化约束参数大小，防止模型过于复杂
- **Top-N 采样**：避免模型重复生成相同内容，增加生成文本的多样性

5 实验总结

本实验通过 PyTorch 实现了一个基于 RNN 的诗歌生成模型，深入理解了：

- 循环神经网络处理序列数据的工作原理
- 字符级语言模型的构建与训练方法
- 嵌入层、Dropout 等组件的实际应用
- TensorBoard 可视化工具的使用
- 困惑度等评价指标的含义

实验表明，两层 RNN 配合合适的超参数设置，能够有效学习诗歌文本的统计规律，生成具有一定古诗风格的文本。使用 TensorBoard 可以直观地观察训练过程中各指标的变化，便于分析模型性能并进行超参数调优。

第 6 节 附录

1 核心代码模块

- 配置管理：Config 类
- 字符转换：TextConverter 类
- 数据集构建：TextDataset 类
- 模型定义：myRNN 类 (Embedding + Dropout + RNN + Linear)
- 生成策略：pick_top_n 采样函数
- 训练记录：SummaryWriter 日志写入

2 模型定义代码

Code Listing 1 RNN 模型定义

```
class myRNN(nn.Module):
    def __init__(self, num_classes, embed_dim, hidden_size,
                  num_layers, dropout):
        super().__init__()
        self.num_layers = num_layers
        self.hidden_size = hidden_size

        self.word_to_vec = nn.Embedding(num_classes, embed_dim)
        self.dropout = nn.Dropout(dropout)
        self.rnn = nn.RNN(embed_dim, hidden_size, num_layers,
                           dropout=dropout if num_layers > 1 else 0)
        self.project = nn.Linear(hidden_size, num_classes)

    def forward(self, x, hs=None):
        batch = x.shape[0]
        if hs is None:
            hs = Variable(
                torch.zeros(self.num_layers, batch, self.hidden_size))
            if use_gpu:
                hs = hs.cuda()
        word_embed = self.word_to_vec(x)
        word_embed = self.dropout(word_embed)
        word_embed = word_embed.permute(1, 0, 2)
        out, h0 = self.rnn(word_embed, hs)
        le, mb, hd = out.shape
        out = out.view(le * mb, hd)
        out = self.project(out)
        out = out.view(le, mb, -1)
        out = out.permute(1, 0, 2).contiguous()
        return out.view(-1, out.shape[2]), h0
```

3 TensorBoard 日志记录

Code Listing 2 TensorBoard 记录

```
# 记录batch损失
writer.add_scalar('Loss/batch', loss.item(), global_step)

# 记录梯度范数
writer.add_scalar('Gradient/norm', total_norm, global_step)

# 记录epoch指标
writer.add_scalar('Loss/epoch', avg_loss, epoch)
```



```
writer.add_scalar('Perplexity', perplexity, epoch)

# 记录学习率
writer.add_scalar('Learning_rate', current_lr, epoch)

# 记录生成的文本
writer.add_text(f'Generated/Epoch_{epoch}', formatted_text, epoch)
```